



618专题（1/2） --大促搭投实践&高可用性保障

 encode-monitor.zip
345.23KB



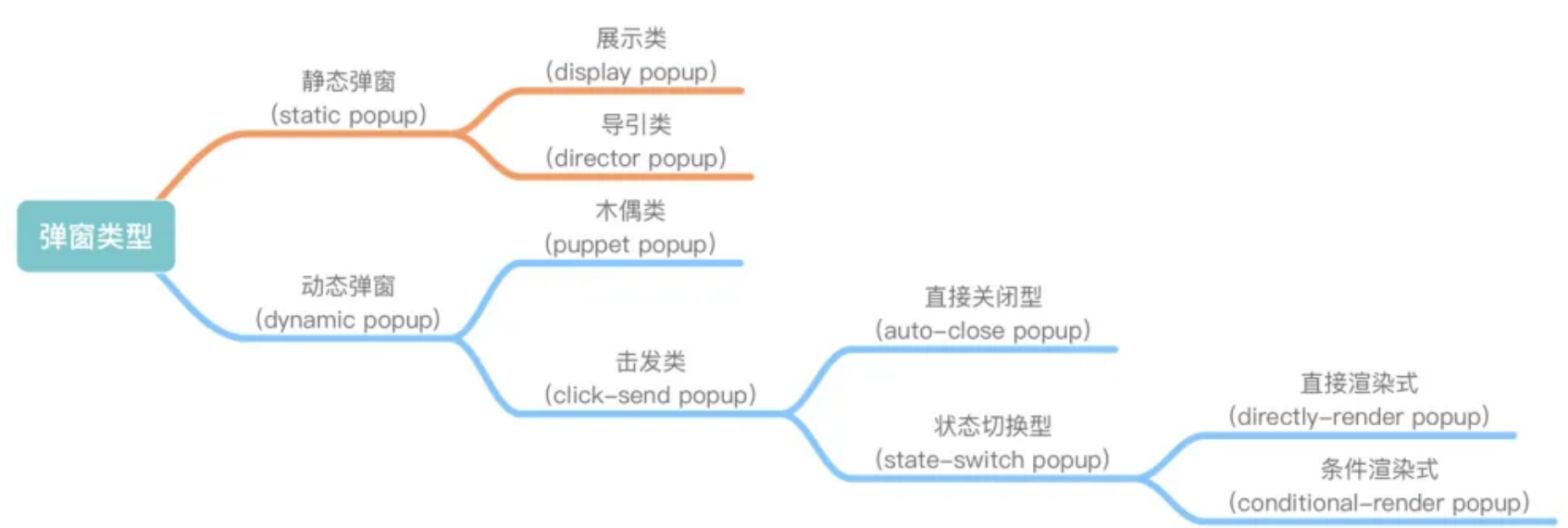
1. 往期部分专题

- [📖 以终为始，重回前端 -- Web Development Trend](#)
- [📖 以终为始，重回前端 --Typescript： JavaScript with syntax for types](#)
- [📖 以终为始，重回前端 -- Awesome JavaScript & Design Patterns](#)
- [📖 以终为始，重回前端（特别篇） -- New React 19 Features You Should Know & How to Upgrade](#)
- [📖 以终为始，重回前端 -- 构建高性能 Vue应用：揭秘 Module Federation 微前端架构的核心技术](#)
- [📖 以终为始，重回前端 -- 揭秘大厂工程化内核，从软件工程视角剖析编译时、运行时及打包构建三板斧](#)
- [📖 以终为始，重回前端 -- 字节前端专家揭秘大厂React最佳实践，手把手从0打造一个开源社区前沿的组件库](#)
- [📖 2025 金三银四面试突击早鸟课 -- 一线25K 前端P6岗的候选人能力要求 & 画像解析](#)
- [📖 2025 金三银四面试突击早鸟课第二弹——字节T2-1岗位面试真题解析](#)
- [📖 2025 金三银四面试突击早鸟课第三弹——手写对标一线城市25K前端P6岗的满分简历](#)
- [📖 金三银四面试突击ALL IN ONE 第三篇 —— 大厂P6级模拟面试来啦](#)
- [📖 金三银四面试突击ALL IN ONE 第四篇 —— 25年金三银四面试高频踩坑解析](#)
- [📖 金三银四面试突击ALL IN ONE 第五篇 —— 金三银四面试利器之前端技术架构设计实操](#)
- [📖 金三银四面试突击ALL IN ONE 第六篇 —— 金三银四大厂真实面试原题解析](#)
- [📖 Vite All in One: Vite工程化开发实践与构建原理解析](#)

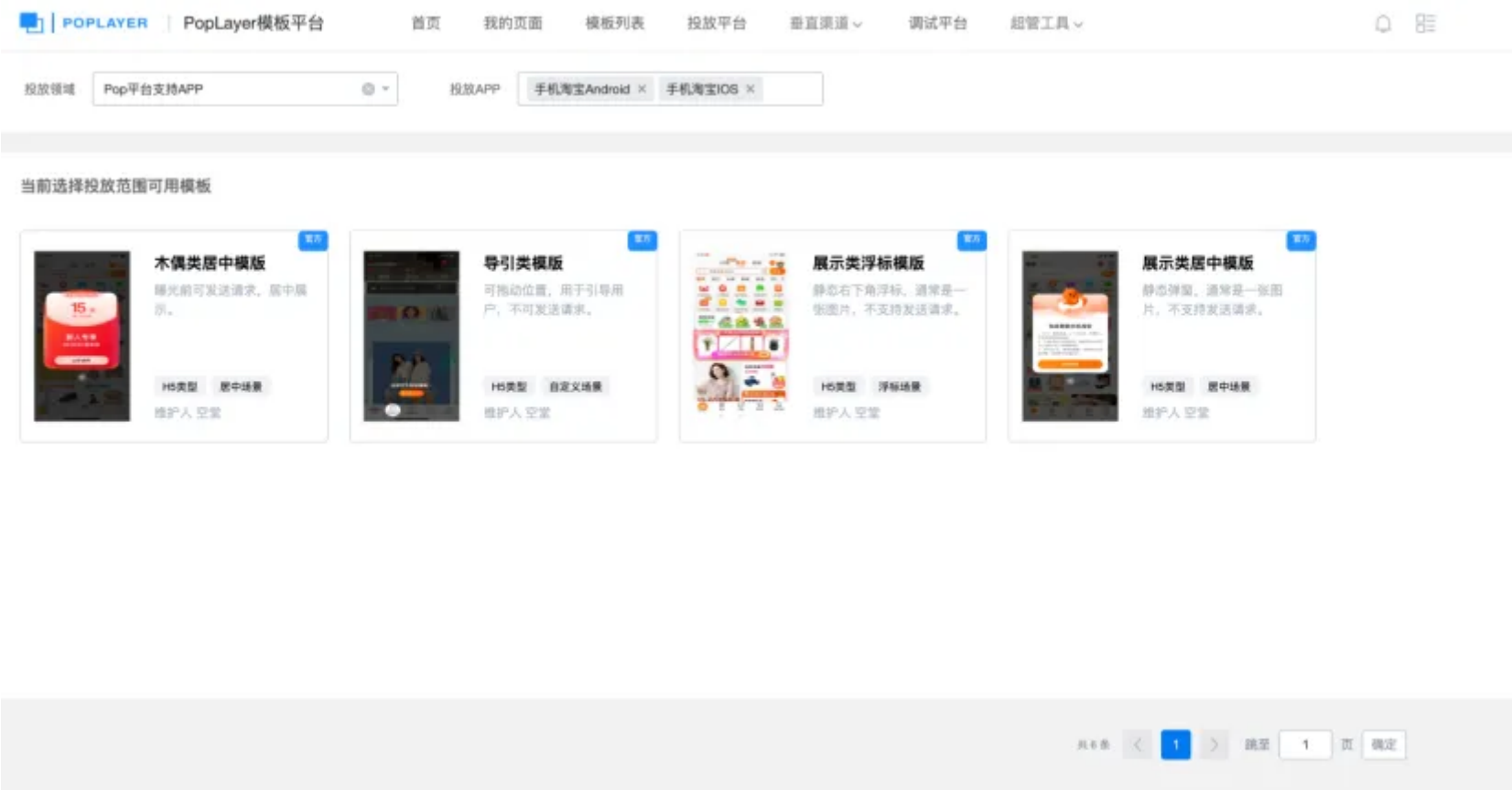
2. 大促弹窗搭投实践

2.1 归纳总结历史弹窗样式功能

想设计一份合理，优雅地 DSL，就首先要对特定领域有深入细致的了解：



通过归纳总结历史弹窗样式，交互和功能，一方面可以设计具备良好扩展性的弹窗描述数据，另一方面，稍后可以看到，**通过为不同类型的弹窗，提供不同类型的弹窗模版，既可以帮助用户提升弹窗搭建效率，也给产品增加了一个有着深远意义的数据维度，可以对比同类型弹窗数据，并从中探索规律促进业务优化。**



2.2 设计新版弹窗描述数据

在搞清楚业务域的领域边界和领域内容后，就可以开始着手进行 DSL 的设计了，这也是本次项目改造升级的核心环节。DSL 设计的好坏，将直接决定整个项目是否有能力在不断发展变化的业务诉求中始终保持稳定和敏捷。

1. 字段设计应符合「奥卡姆剃刀」原则，如非必要，勿增实体：尽可能让数据字段精简，明确，同时，数据组织要合理；
2. 数据模块之间应满足「高内聚，低耦合」原则，尽量避免修改某一个字段时，牵一发而动全身，导致大面积的字段修改；
3. 需紧密围绕特定领域进行设计，既要避免过度设计导致数据结构复杂和字段膨胀，但同时也要充分考虑到未来可能出现的种种可能；理想情况下，未来发展出的新的业务诉求，只需「更新」DSL 的属性值，而非属性；

基于以上三个核心设计原则，最终产出了淘宝弹窗描述数据规范。其大致可以分为如下 7 个模块

- 1. **基础信息模块**：包含弹窗的基本信息，如名称，类型等；
- 2. **组件模块**：包含弹窗域所需组件的描述，通过分析历年弹窗域活动内容，归纳总结出**图片，文字，倒计时，热区，容器，iframe，视频，状态和关闭按钮**共 9 种组件，每种组件都围绕弹窗域需求与手淘基础架构设置了特定的字段，例如对于文字组件，可设置行数与内容，对于倒计时组件可设置是否显示毫秒位，倒计时结束后行为，超过某个临界值是否显示文本等；定制化组件使用户可以非常直观便利地使用各组件完成弹窗搭建任务；
- 3. **样式模块**：负责描述组件的 UI，由于不确定终端渲染环境，因此在样式属性，样式单位的设计上就要充分考虑到不同场景，不同技术选型的限制。通过与客户端同学沟通交流，最终将样式属性确定为能满足三端（H5，iOS，Android）与弹窗需要的最小样式集合；
- 4. **请求模块**：请求模块涵盖了对手淘内常见请求方式的结构化描述，并将该描述同时使用在弹窗曝光前与曝光后，数据内容覆盖了弹窗域请求所需关注的所有关注点，并支持串行，并行请求的配置；

1. 弹窗描述数据：TPDD（Taobao Popup Description Data）

一份符合 TPDD 规范的数据包含了弹窗渲染所需的全部数据。一级字段共 17 个，可以分为四类：

- 1. **基础数据**（5个）：`activityId`，`name`（可选），`description`（可选），`tag`（可选），`type`（可选）；
- 2. **请求数据**（3个）：`store`，`prefetch`，`request`（可选）；
- 3. **组件数据**（1个）：`component`；
- 4. **配置数据**（6个）：`condition`，`countTime`，`neverShow`，`spm`（可选），`defaultToastText`（可选），`autoCloseTime`（可选）；
- 5. **附加功能**（2个）：`displayMode`（可选），`preloadUrl`（可选）；

2.2.1 行为模块的设计

行为模块描述弹窗对用户行为的响应，在数据结构的设计上，站在被响应物体的角度，描述了物体应响应什么，响应的目标以及响应的内容，具体字段设计如下：

- `type`：表示响应事件的类型，如点击（click），Tab 切换（tabSwitch）等；
- `behavior`：表示弹窗的**行为类别**，经过长时间的归纳总结，可将弹窗行为分类为：
 - `goto`：页面跳转；
 - `switch`：弹窗内部状态切换；
 - `call`：端外环境唤端至端内；
 - `request`：发送请求；
 - `close`：关闭弹窗；
 - `doNothing`：什么也不做（默认）；
 - `show`：显示；
 - `hide`；隐藏；
- `target`：表示弹窗**行为对象**，会根据 `behavior` 字段的不同而拥有不同语义，例如当 `behavior` 为 `switch` 时，`target` 表示待切换的状态 ID；
- `content`：表示弹窗**行为的内容**，例如当 `behavior` 值为 `goto`，`target` 值为 `app`，`content` 值为 <https://www.taobao.com> 时，表明事件发生时，页面需跳转至端内首页；

通过这样的数据结构设计，可以最大限度的清晰表达弹窗的行为，并通过基础字段的扩展和组合，在不改变数据结构的情况下灵活，高效地支持各式各样的新需求。

一般而言，弹窗针对用户行为仅会有一次响应。但随着对淘宝场景，用户行为感知能力的不断提示，在极少数场景下开始出现需要弹窗响应多个行为的需求。在权衡场景的稀有性和 DSL 的扩展性后，针对这种特殊场景开辟了 actions 字段，数据结构如下：

```
{
  actions:[
    {
      "type": "click",
      "content": [
        {
          "id": "1",
          "target": "State_2",
          "behaivior": "switch",
          "content": "",
          "nextActions": ["2"],
          "fallbackActions": []
        },
        {
          "id": "2",
          "target": "",
          "behaivior": "close",
          "content": "",
          "nextActions": [],
          "fallbackActions": [],
        }
      ]
    }
  ]
}
```

2.3 DSL维护

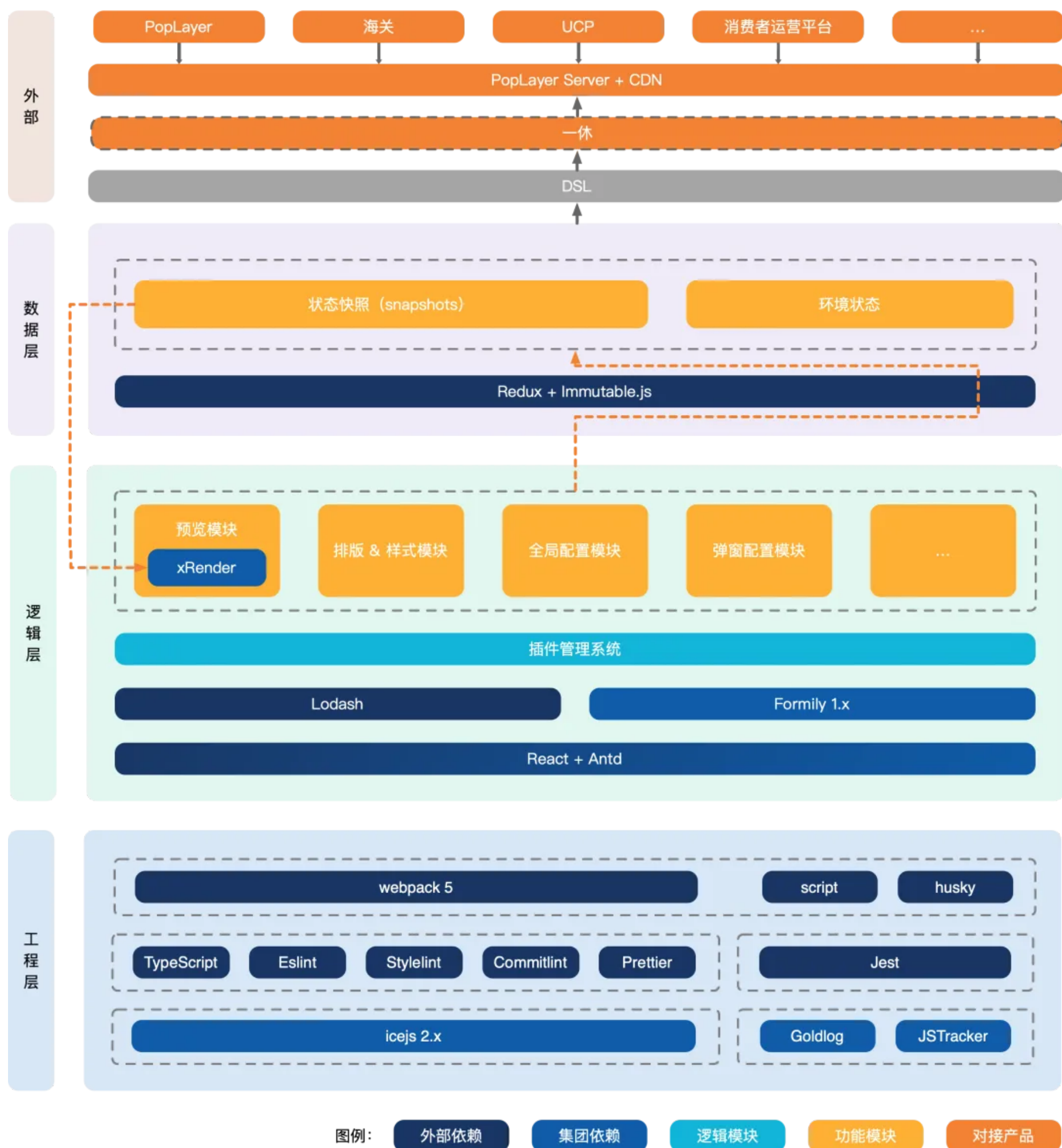
值得一提的是，在设计好一个有着良好架构的 DSL 只不过是完成了万里长征的第一步。随着业务的不断发展，开发者对领域知识的更加深入地理解。DSL 当前的结构总会面临着新的挑战，此时，设计者的核心责任则在于，严格的把控 DSL 的复杂度，坚持仅在必要的时候才会对 DSL 的结构做合理的，小幅的修改。

如果设计者在漫长的维护岁月中逐渐丧失定力，那么这份一开始精心设计的 DSL，最终也会像绝大多数糟糕设计一样，迅速膨胀，变得丑陋，难以理解，不易维护。所以设计者应该始终保持高度的责任感，完成如下事宜：

1撰写细致，友好的文档，并定时更新；2文档应及时标明某些字段的特殊性，如已废弃或支持不完全，避免其他开发者踩坑；**3在涉及 DSL 的变更时，需要严肃思考变更的合理性，变更方式和潜在隐患**，谨小慎微，科学规范的进行迭代；

以上，便是对本次产品升级的核心 -- DSL 重新设计的一些经验分享，在确定好一份可以清晰描述各类弹窗，具备良好扩展性的数据之后，下一步工作则是提供一个面向用户，用于产出满足数据规范数据的弹窗搭建平台。

下方是弹窗编辑器 xEditor 的技术架构图：



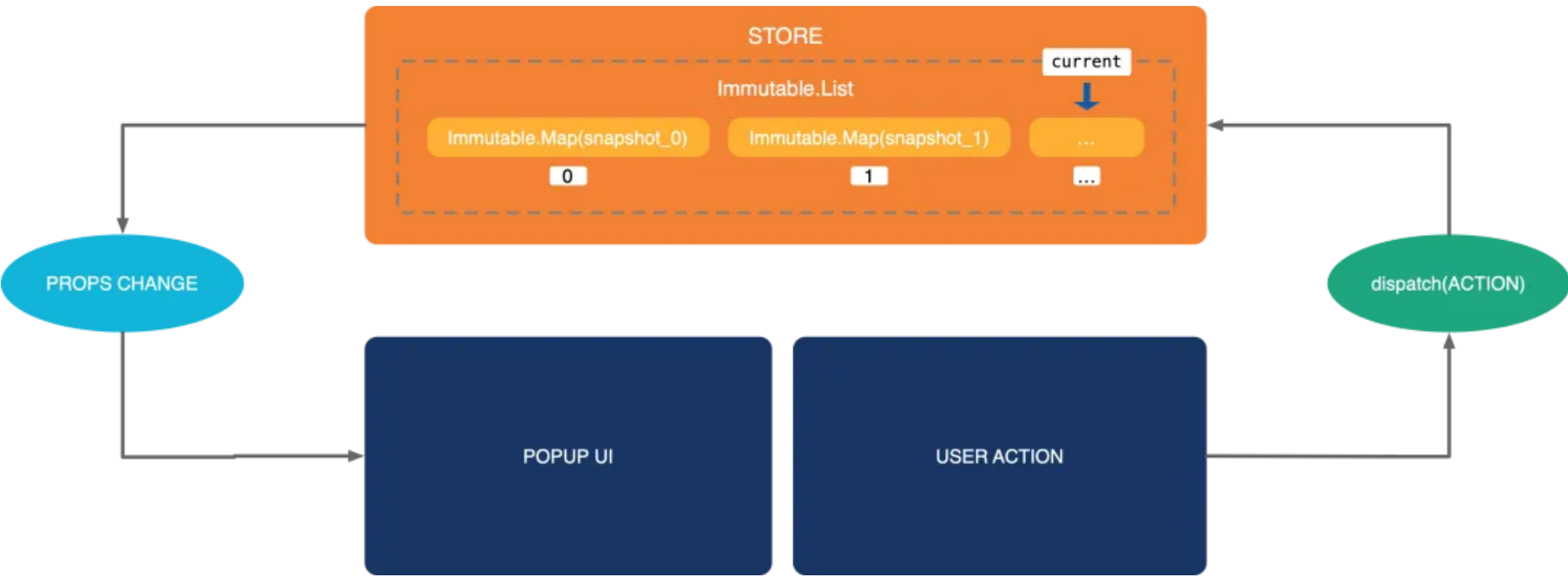
从架构图中可以清晰地看到，xEditor 的整体架构可以分为**数据层**，**逻辑层**部分，要想让整个应用始终保持稳定性与高扩展性，在每层做了如下设计：

2.3.1 数据层设计

数据层主要包含编辑器内部状态的管理以及对外的数据交互。由于编辑器的功能在于产出满足 DSL 规范的数据，因此对外的数据交互较为简单，核心应关注内部状态的管理。

为了保障状态变更的有序性和可维护性，通过 immutable.js 将用户的每一次操作结果保存为一个「**数据快照**」，并压入栈中。由于每一个快照都是一份不可变的满足 DSL 规范的数据，因此快照的每一次更新都可以促使预览模块即时渲染出最新的弹窗样式。**从而**

通过严格的数据驱动渲染使整个应用的状态井然有序。并且由于预览模块采用了和线上一致的渲染引擎，因此还可以真正实现「所见即所得」的效果



同时，由于用户的每次操作结构都被保存下来，因此整个应用还可以轻松实现用户行为记录，回退功能，进一步提升用户体验。

2.3.2 逻辑层设计

在逻辑层要解决的核心问题是让应用具备灵活的扩展能力，为此，将整个用户界面按照功能模块拆分成一个个有标准化接口的组件，并通过在底层实现一个有标准化「插槽」的 Layout 基座，使得每个功能模块都能以类似插件的方式，通过插拔形式拼装出一个具备完整功能的应用界面。

```
<Spin
  size='large'
  spinning={isEmpty(SNAPSHOT)}
  tip='数据获取中...'
  wrapperClassName={styles.spinWrapper}
>
  {/* 顶部工具栏: 多组件组合, 排版等 */}
  <Header />
  {/* 顶部样式栏: 文字样式, 尺寸, 位置等 */}
  <StyleOptBar
    snapshots={snapshots}
    optTarget={OPT_TARGET as ICDD}
    updateFragmentFunc={updateFragmentFunc}
  />
  {/* 预览区 & 操作区 */}
  <OperationArea />
  {/* 插件 */}
  <Plugins
    optTarget={OPT_TARGET}
    updateFragmentFunc={updateFragmentFunc}
  />
  {/* 超管工具 */}
  <SuperAdminTools snapshot={SNAPSHOT} saveSnapshot={saveSnapshot} />
</Spin>
```

通过这样的设计，使得每个功能区块均彼此独立，且可根据需求迅速扩展或移除，开发者不必担心会影响其他模块功能，并且应用还可以根据不同用户区分所要对外暴露的功能。

3. 完整的前端技术栈应该掌握到什么程度？



EncodeStudio
印客学院

印客学院2025匠心之作
内容&服务全面升级

WEB

Web前端

大厂工程师训练营

进阶前端架构
直击阿里P7



4. 大促高可用性保障

4.1 怎样的系统才是稳定的？

Google SRE中(SRE三部曲[1])有一个层级模型来描述系统可靠性基础和高层次需求(Dickerson's Hierarchy of Service Reliability)，如下图：

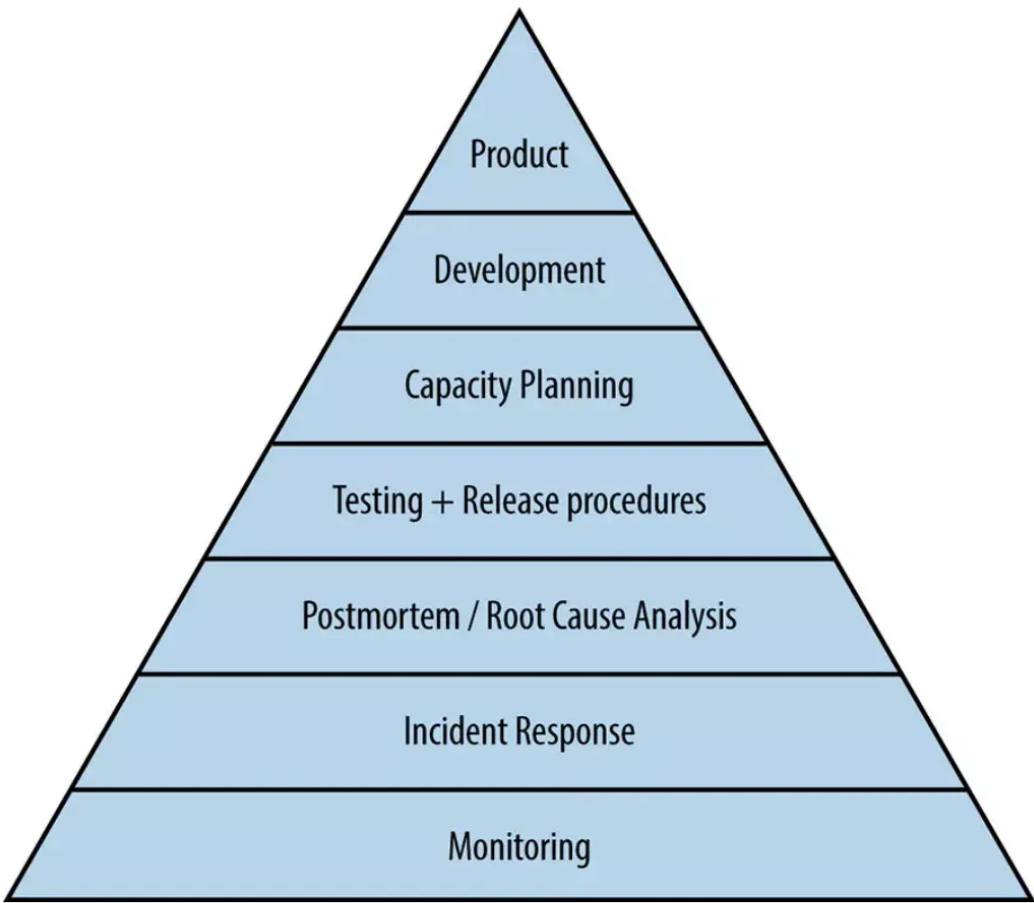


Figure III-1. Service Reliability Hierarchy

该模型由Google SRE工程师Mikey Dickerson在2013年提出，将系统稳定性需求按照基础程度进行了不同层次的体系化区分，形成稳定性标准金字塔模型。

金字塔的底座是监控(Monitoring)，这是一个系统对于稳定性最基础的要求，缺少监控的系统，如同蒙上眼睛狂奔的野马，无从谈及可控性，更遑论稳定性。更上层是应急响应(Incident Response)，从一个问题被监控发现到最终解决，这期间的耗时直接取决于应急响应机制的成熟度。合理的应急策略能保证当故障发生时，所有问题能得到有序且妥善的处理，而不是慌乱成一锅粥。事后总结以及根因分析(Postmortem&Root Caue Analysis)，即我们平时谈到的“复盘”，虽然很多人都不太喜欢这项活动，但是不得不承认这是避免我们下次犯同样错误的最有效手段，只有当摸清故障的根因以及对应的缺陷，我们才能对症下药，合理进行规避。

假设一个系统从初次发布后就不再进行更新迭代，做好上述三个方面的工作就能基本满足系统对于稳定性的全部需求。可惜目前基本不会存在这样的系统，大大小小的应用都离不开不断的变更与发布，因此要保证系统在这些迭代中持续稳定，测试和发布管控(Testing&Release procedures)是必不可少的。有效的测试与发布策略能保障系统所有新增变量都处于可控稳定区间内，从而达到整体服务终态稳定。除了代码逻辑更新，迭代同样可能带来业务规模及流量的变化，容量规划(Capacity Planning)则是针对于这方面变化进行的保障策略。现有系统体量是否足够支撑新的流量需求，整体链路上是否存在不对等的薄弱节点，都是容量规划需要考虑的问题。

位于金字塔模型最顶端的是产品设计(Product)与软件研发(Development)，即通过优秀的产品设计与软件设计使系统具备更高的可靠性，构建高可用产品架构体系，从而提升用户体验。

4.2 大促稳定性保障方法

从金字塔模型我们可以看到构建维护一个高可用服务所需要做到的几方面工作，那么问题回到大促稳定性，如何体系化地保障大促期间系统稳定性？

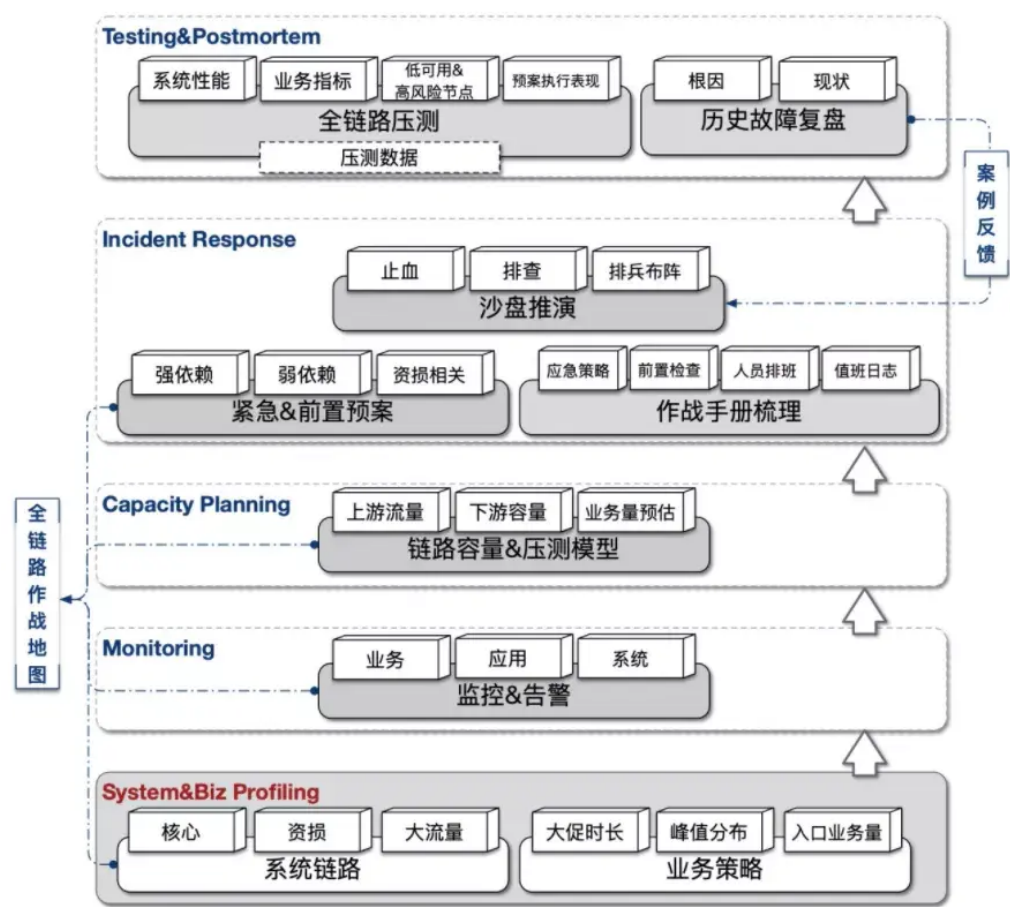
大促保障实际上针对于特定业务场景的集中稳定性建设工作，相较于日常保障工作，具有高并发流量、短保障周期的特点，对系统性能与保障时间有明确要求（一般为2个月左右）。

考虑到上述特性，我们如何在短时间内针对大促大流量业务场景对系统稳定性需求进行优化巩固？

既然时间有限，盲目撒网必然不是最佳策略，需要有针对性地从关键点、薄弱点下手。因此第一步，需要获得全局系统链路现状，包括关键外部依赖、重点业务影响等，找到整体保障的核心关注点。接下来进一步分析大促业务数据，得到除系统本身以外的变量干扰因素。以这两者为基础，集中围绕金字塔模型中系统监控、规划容量、应急响应、测试和复盘等几个方面需求对系统进行针对性集中保障建设，得到最终保障结果。

至此，基本获得了完整的大促稳定性保障策略方向，按照执行顺序依次是：

- 1. 系统链路&业务策略梳理（System & Biz Profiling）
- 2. 监控（Monitoring）
- 3. 容量规划（Capacity Planning）
- 4. 应急响应（Incident Response）
- 5. 测试
- 6. 事后总结（Testing & Postmortem）



4.2.1 系统链路梳理

4.2.1.1 入口梳理盘点

一个系统往往存在十几个甚至更多流量入口，包含HTTP、RPC、消息等都多种来源。如果无法覆盖所有所有链路，可以从以下三类入口开始进行梳理：

- 核心重保流量入口用户承诺服务SLI较高，对数据准确性、服务响应时间、可靠度具有明确要求。面向企业级用户
- 资损事件对应入口关联到公司资金收入或者客户资金收入收费服务
- 大流量入口系统TPS&QPS TOP5~10该类入口虽然不涉及较高SLI与资损要求，但是流量较高，对整体系统负载有较大影响。

4.2.1.2 节点分层判断

流量入口就如同线团中的线头，挑出线头后就可按照流量轨迹对链路上的节点(HSFDBTairHBase等一切外部依赖)按照依赖程度、可用性、可靠性进行初级分层区分。

(1) 强弱依赖节点判断

- 若节点不可用，链路业务逻辑被中断 or 高级别有损(存在一定耐受阈值)，则为业务强依赖；反之为弱依赖。
- 若节点不可用，链路执行逻辑被中断(return error)，则为系统强依赖；反之为弱依赖。
- 若节点不可用，系统性能受影响，则为系统强依赖；反之为弱依赖。按照快速失败设计逻辑，该类节点不应存在，但是在不变更应用代码前提下，如果出现该类节点，应作为强依赖看待。

- 若节点无感可降级 or 存在业务轻微损伤替换方案，则为弱依赖。

(2) 低可用依赖节点判断

- 节点服务日常超时严重
- 节点对应系统资源不足

(3) 高风险节点判断

- 上次大促后，节点存在大版本系统改造
- 新上线未经历过大促的节点
- 节点对应系统是否曾经出现高级别故障
- 节点故障后存在资损风险

4.2.1.3 入口梳理盘点

一个系统往往存在十几个甚至更多流量入口，包含HTTP、RPC、消息等都多种来源。如果无法覆盖所有所有链路，可以从以下三类入口开始进行梳理：

- 核心重保流量入口用户承诺服务SLI较高，对数据准确性、服务响应时间、可靠度具有明确要求。面向企业级用户
- 资损事件对应入口关联到公司资金收入或者客户资金收入收费服务
- 大流量入口系统TPS&QPS TOP5~10该类入口虽然不涉及较高SLI与资损要求，但是流量较高，对整体系统负载有较大影响。

4.2.2 业务策略同步

不同于高可用系统建设体系，大促稳定性保障体系与面向特定业务活动的针对性保障建设，因此，业务策略与数据是我们进行保障前不可或缺的数据。

一般大促业务数据可分为两类，全局业务形态评估以及应急策略&玩法。

4.2.2.1 全局评估

该类数据从可以帮助我们进行精准流量评估、峰值预测、大促人力排班等等，一般包含下面几类：

- 大促业务时长（XX日-XX日）
- 业务量预估体量（日常X倍）
- 预估峰值日期
- 业务场景预估流量分配

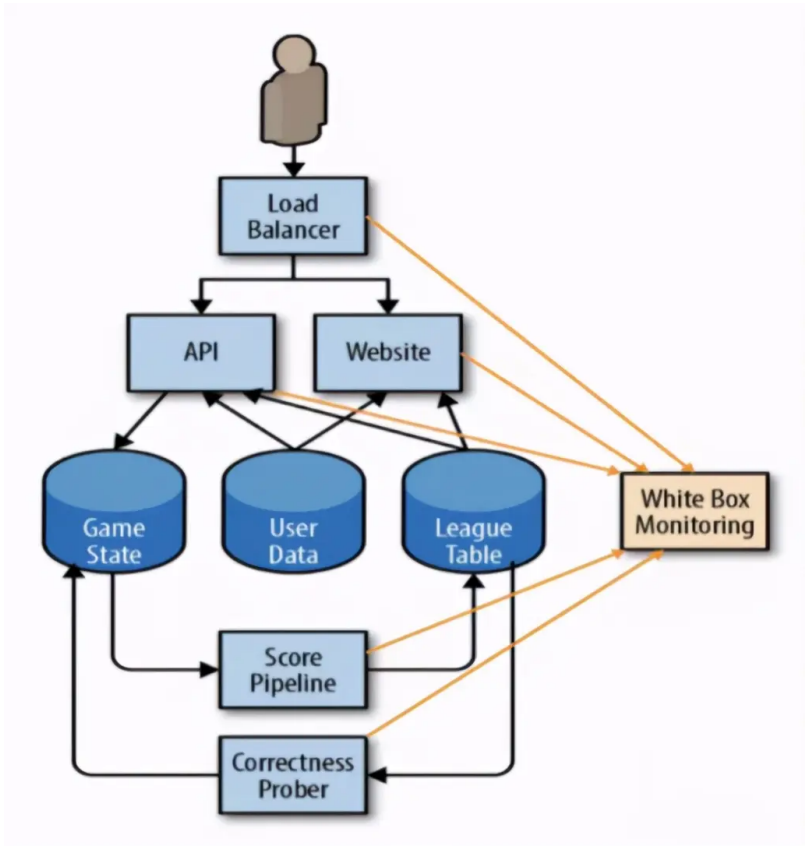
4.2.2.2 应急策略

该类数据指相较于往年大促活动，本次大促业务变量，可用于应急响应预案与高风险节点评估等，一般包含下面两类：

- 特殊业务玩法
- 应急情况打法策略

4.2.3 Monitoring - 监控&告警梳理

目前业界常用的监控手段一般有两种模式，黑盒监控(Black-box monitoring)与白盒监控(White-box monitoring)。黑盒监控面向对象，一般监控正在发生（而非即将发生）的异常，即系统现有故障。而白盒监控主要依赖系统内部指标监控，面向对象的同时也面向原因，可对系统即将面临的异常进行提前预警，也可在异常发生时同步监控下层内部指标，从而定位根本原因。因此大促稳定性保障中，我们一般选择的是白盒监控。



站在监控的角度看，我们的系统从上到下一般可以分为三层：业务（Biz）、应用（Application）、系统（System）。系统层为最下层基础，表示操作系统相关状态；应用层为JVM层，涵盖主应用进程与中间件运行状态；业务层为最上层，为业务视角下服务对外运行状态。

因此进行大促稳定性监控梳理时，可以先脱离现有监控，先从核心、资损链路开始，按照业务、应用（中间件、JVM、DB）、系统三个层次梳理需要哪些监控，再从根据这些索引找到对应的监控告警，如果不存在，则相应补上；如果存在则检查阈值、时间、告警人是否合理。

4.2.3.1 监控

监控系统一般有四项黄金指标：延时（Latency），错误（Error），流量（Traffic），饱和度（Situation），各层的关键性监控同样也可以按照这四项指标来进行归类，具体如下：

	业务层 (Biz-level)	应用层 - 中间件 (Application-level)	应用层 - JVM (Application-level)	系统层 (System-level)
Latency	核心入口响应时间	读 \ 写RT (DataSource Group \ Atom \ Table)	VM线程数	I/O wait
Traffic	核心入口TPS/QPS	TPS \ QPS (DataSource Group \ Atom \ Table)	OOM	- CPU使用率 - 内存使用率 - TCP 流量 - 线程数
Errors	- 关键性错误监控 - TOP错误监控	- TDDL层错误 (eg:Druid连接池满) - DB Servicer层错误 (eg:主键冲突) - RPC Client&Service 线程池满	- 堆内、外内存使用率 - YGC、FGC时间、频率	- 主进程消失 - TCP retry - SSH断开连接
Situation	- 业务成功率 - 业务数据正确率 - 业务关注指标(业务大盘)	- 读 \写成功率 - DB connection pool在用连接数 - HSF超时率		Load

4.2.3.2 告警

是不是每项监控都需要告警？答案当然是否定的。建议优先设置Biz层告警，因为Biz层我们对外服务最直观业务表现，最贴切用户感受。Application&System层指标主要用于监控，部分关键&高风险指标可设置告警，用于问题排查定位以及故障提前发现。

对于一项告警，我们一般需要关注级别、阈值、通知人等几个点。

1. 级别

即当前告警被触发时，问题的严重程度，一般来说有几个衡量点：

- 是否关联GOC

- 是否产生严重业务影响
- 是否产生资损

2. 阈值

即一项告警的触发条件&时间，需根据具体场景合理制定。一般遵循以下原则：

- 不可过于迟钝。一个合理的监控体系中，任何异常发生后都应触发相关告警。
- 不可过于敏感。过于敏感的阈值会造成频繁告警，从而导致响应人员疲劳应对，无法筛选真实异常。若一个告警频繁出现，一般是两个原因：系统设计不合理 or 阈值设置不合理。
- 若单一指标无法反馈覆盖整体业务场景，可结合多项指标关联构建。
- 需符合业务波动曲线，不同时段可设置不同条件&通知策略。

3. 通知人&方式

若为业务指标异常(Biz层告警)，通知人应为问题处理人员(开发、运维同学)与业务关注人员(TL、业务同学)的集合，通知方式较为实时，比如电话通知。

若为应用 & 系统层告警，主要用于定位异常原因，通知人设置问题排查处理人员即可，通知方式可考虑钉钉、短信等低干扰方式。

除了关联层次，对于不同级别的告警，通知人范围也可适当扩大，尤其是关联GOC故障的告警指标，应适当放宽范围，通知方式也应更为实时直接。

4.2.3.3 应产出数据

完成该项梳理工作后，我们应该产出以下数据：

- 系统监控模型，格式同表1Biz、Application、System 分别存在哪些待监控点监控点是否已全部存在指标，仍有哪些待补充
- 系统告警模型列表，需包含以下数据关联监控指标（链接）告警关键级别是否推送GOC是否产生资损是否关联故障是否关联预案
- 业务指标大盘，包含Biz层重点监控指标数据。
- 系统&应用指标大盘，包含核心系统关键系统指标，可用于白盒监控定位问题。

4.2.4 紧急&前置预案梳理

要想在大促高并发流量场景下快速对线上紧急事故进行响应处理，仅仅依赖值班同学临场发挥是远远不够的。争分夺秒的情况下，无法给处理人员留有充足的策略思考空间，而错误的处理决策，往往会导致更为失控严重的业务&系统影响。因此，要想在大促现场快速而正确的响应问题，值班同学需要做的是选择题(Which)，而不是陈述题(What)。而选项的构成，便是我们的业务&系统预案。

从执行时机与解决问题属性来划分，预案可分为技术应急预案、技术前置预案、业务应急预案、业务前置预案等四大类。结合之前的链路梳理和业务评估结果，我们可以快速分析出链路中需要的预案，遵循以下原则：

- 技术应急预案：该类预案用于处理系统链路中，某层次节点不可用的情况，例如技术/业务强依赖、弱稳定性、高风险等节点不可用等异常场景。
- 技术前置预案：该类预案用于平衡整体系统风险与单节点服务可用性，通过熔断等策略保障全局服务可靠。例如弱稳定性&弱依赖服务提前降级、与峰值流量时间冲突的离线任务提前暂定等。
- 业务应急预案：该类预案用于应对业务变更等非系统性异常带来的需应急处理问题，例如业务数据错误（数据正确性敏感节点）、务策略调整（配合业务应急策略）等
- 业务前置预案：该类预案用于配合业务全局策略进行的前置服务调整（非系统性需求）

4.2.4.1 应产出数据

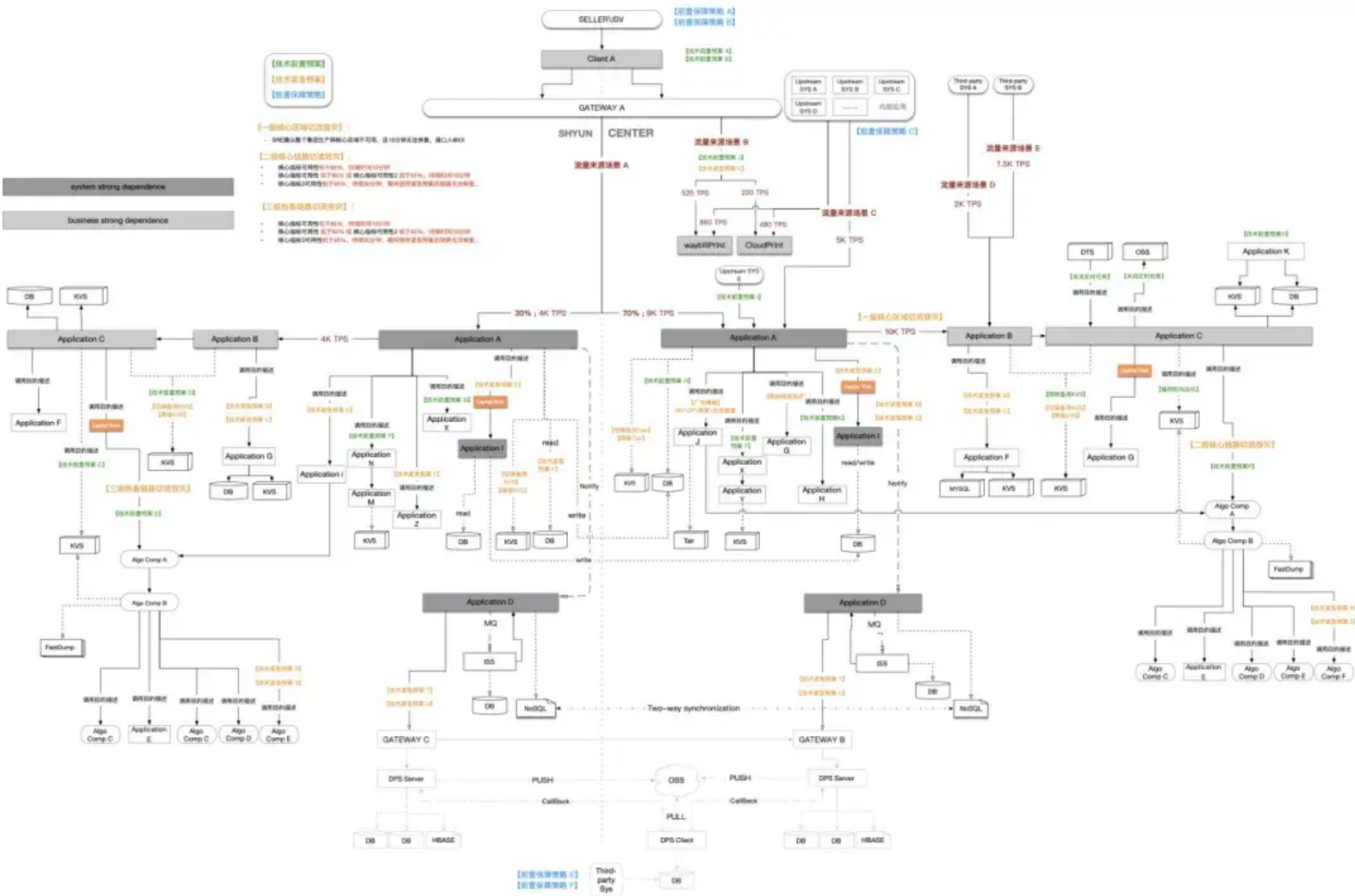
完成该项梳理工作后，我们应该产出以下数据：

- 执行&关闭时间（前置预案）
- 触发阈值（紧急预案，须关联相关告警）
- 关联影响（系统&业务）
- 决策&执行&验证人员
- 开启验证方式

- 关闭阈值（紧急预案）
- 关闭验证方式

4.2.5 阶段性产出-全链路作战地图

进行完上述几项保障工作，我们基本可得到全局链路作战地图，包含链路分支流量模型、强弱依赖节点、资损评估、对应预案&处理策略等信息。大促期间可凭借该地图快速从全局视角查看应急事件相关影响，同时也可根据地图反向评估预案、容量等梳理是否完善合理。



4.2.6 作战手册梳理

作战手册是整个大促保障的行动依据，贯穿于整个大促生命周期，可从事前、事中、事后三个阶段展开考虑。整体梳理应本着精准化、精细化的原则，理想状态下，即便是对业务、系统不熟悉的轮班同学，凭借手册也能快速响应处理线上问题。

4.2.6.1 事前

1. 前置检查事项清单

大促前必须执行事项checklist,通常包含以下事项：

- 集群机器重启 or 手动FGC
- 影子表数据清理
- 检查上下游机器权限
- 检查限流值
- 检查机器开关一致性
- 检查数据库配置
- 检查中间件容量、配置(DB缓存NoSQL等)
- 检查监控有效性（业务大盘、技术大盘、核心告警）

- 每个事项都需包含具体执行人、检查方案、检查结果三列数据

2. 前置预案

域内所有业务&技术前置预案。

4.2.6.2 事中

1. 紧急技术&业务预案

需要包含的内容基本同前置预案，差异点如下：

- 执行条件&恢复条件：具体触发阈值，对应监控告警项。
- 通知决策人。

2. 应急工具&脚本

常见故障排查方式、核心告警止血方式(强弱依赖不可用等)，业务相关日志捞取脚本等。

3. 告警&大盘

应包含业务、系统集群及中间件告警监控梳理结果，核心业务以及系统大盘，对应日志数据源明细等数据：

- 日志数据源明细：数据源名称、文件位置、样例、切分格式。
- 业务、系统集群及中间件告警监控梳理结果：关联监控指标（链接）、告警关键级别、是否推送GOC、是否产生资损、是否关联故障、是否关联预案。
- 核心业务&系统大盘：大盘地址、包含指标明细(含义、是否关联告警、对应日志)。

4. 上下游机器分组

应包含核心系统、上下游系统，在不同机房、单元集群分组、应用名，可用于事前-机器权限检查、事中-应急问题排查黑屏处理。

5. 值班注意事项

包含每班轮班同学值班必做事项、应急变更流程、核心大盘链接等。

6. 核心播报指标

包含核心系统&服务指标(CPULOADRT)、业务关注指标等，每项指标应明确具体监控地址、采集方式。

7. 域内&关联域人员通讯录、值班

包含域内技术、TL、业务方对应排班情况、联系方式(电话)，相关上下游、基础组件(DB、中间件等)对应值班情况。

8. 值班问题记录

作战记录，记录工单、业务问题、预案(前置紧急)（至少包含：时间、问题描述（截图）、影响分析、决策&解决过程等）。值班同学在值班结束前，进行记录。

4.2.6.3 事后

1. 系统恢复设置事项清单(限流、缩容)

一般与事前检查事项清单对应，包含限流阈值调整、集群缩容等大促后恢复操作。

2. 大促问题复盘记录

应包含大促遇到的核心事件总结梳理。

4.2.7 沙盘推演

实战沙盘演练是应急响应方面的最后一项保障工作，以历史真实故障CASE作为应急场景输入，模拟大促期间紧急状况，旨在考验值班同学们对应急问题处理的响应情况。

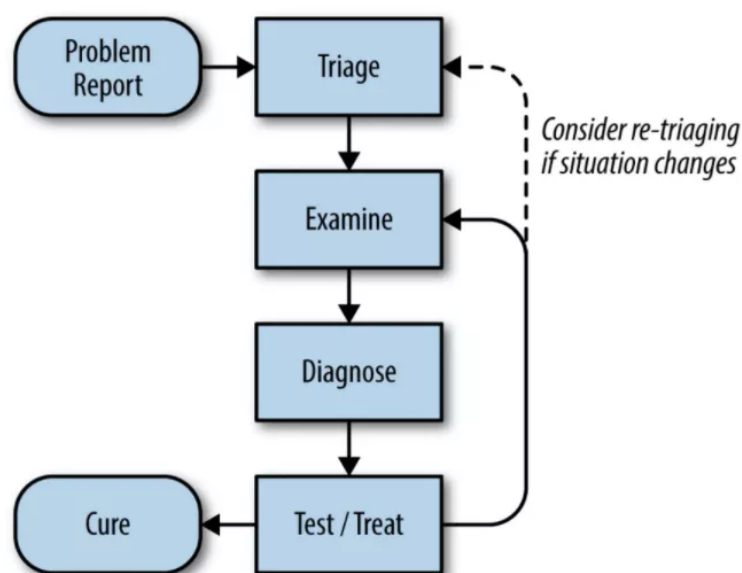
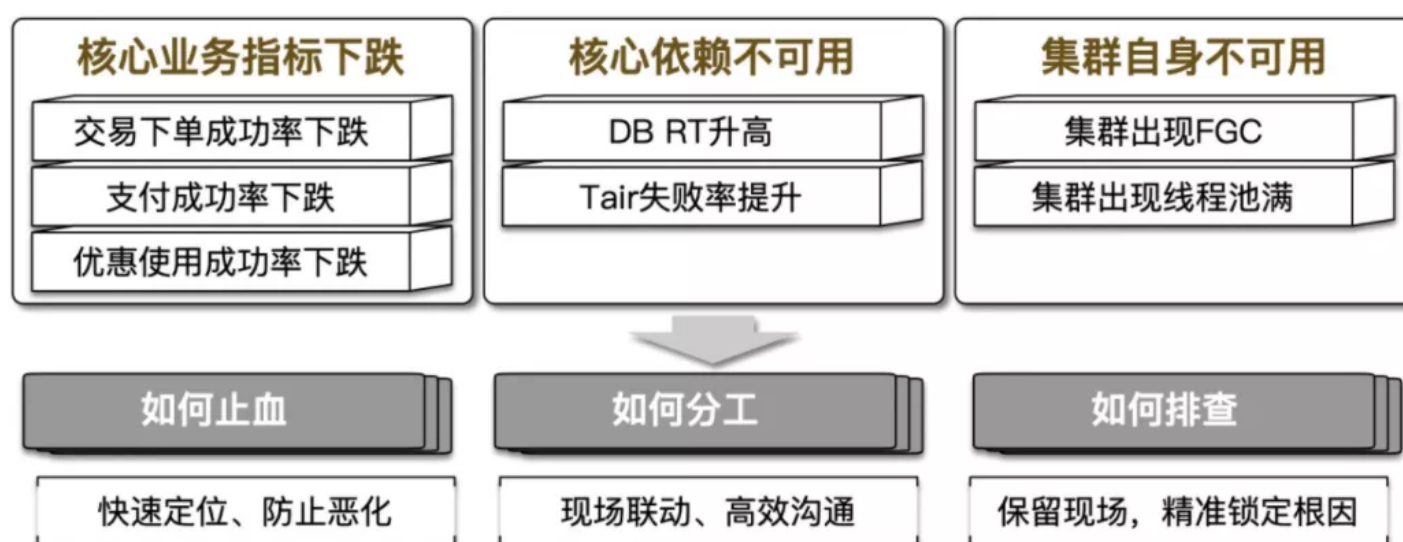


Figure 12-1. A process for troubleshooting

一般来说，一个线上问题从发现到解决，中间需要经历定位&排查&诊断&修复等过程，总体遵循以下几点原则：

- 尽最大可能让系统先恢复服务，同时为根源调查保护现场（机器、日志、水位记录）。
- 避免盲目搜索，依据白盒监控针对性诊断定位。
- 有序分工，各司其职，避免一窝蜂失控乱象。
- 依据现场情况实时评估影响范围，实在无法通过技术手段挽救的情况(例如强依赖不可用)，转化为业务问题思考（影响范围、程度、是否有资损、如何协同业务方）。
- 沙盘演练旨在检验值班同学故障处理能力，着重关注止血策略、分工安排、问题定位等三个方面：



国际化中台双11买家域演练

根据故障类型，常见止血策略有以下解决思路：

- 入口限流：调低对应Provider服务来源限流值应对突发流量过高导致自身系统、下游强依赖负载被打满。
- 下游降级：降级对应下游服务下游弱依赖不可用。下游业务强依赖经业务同意后降级（业务部分有损）。
- 单点失败移除：摘除不可用节点单机水位飙升时，先下线不可用单机服务（无需下线机器，保留现场）。应对集群单点不可用、性能差。
- 切换：单元切流或者切换备份应对单库或某单元依赖因为自身原因（宿主机或网络），造成局部流量成功率下跌下跌。

Google SRE中，对于紧急事故管理有以下几点要素：

- 嵌套式职责分离，即分确的职能分工安排
- 控制中心作战室
- 实时事故状态文档
- 明确公开的职责交接

其中嵌套式职责分离，即分确的职能分工安排，达到各司其职，有序处理的效果，一般可分为下列几个角色：

- 事故总控：负责协调分工以及未分配事务兜底工作，掌握全局概要信息，一般为PM/TL担任。
- 事务处理团队：事故真正处理人员，可根据具体业务场景&系统特性分为多个小团队。团队内部存在域内负责人，与事故总控人员进行沟通。
- 发言人：事故对外联络人员，负责对事故处理内部成员以及外部关注人员信息做周期性信息同步，同时需要实时维护更新事故文档。
- 规划负责人：负责外部持续性支持工作，比如当大型故障出现，多轮排班轮转时，负责组织职责交接记录。

4.3 前端监控常见方案

1. `encode-monitor-browser`：前端稳定性监控 页面监控；
2. `encode-monitor-core`：前端稳定性监控 核心功能；
3. `encode-monitor-react`：前端稳定性监控 React 监控；
4. `encode-monitor-vue`：前端稳定性监控 Vue 监控；
5. `encode-monitor-web`：前端稳定性监控 Web 监控；
6. `encode-monitor-web-performance`：前端稳定性监控 Web 性能监控；
7. `encode-monitor-wx-mini-program`：前端稳定性监控 小程序监控；
8. `encode-monitor-wx-mini-program-performance`：前端稳定性监控 小程序性能监控；

- 全局错误捕获：`window.onError`；
- 请求错误拦截：`XMLHttpRequest`；
- Promise异常捕获：`unhandledrejection`；
- React错误捕获：`componentDidCatch`；
- Vue错误捕获：`errorHandler`；
- 用户行为信息：`performance`；